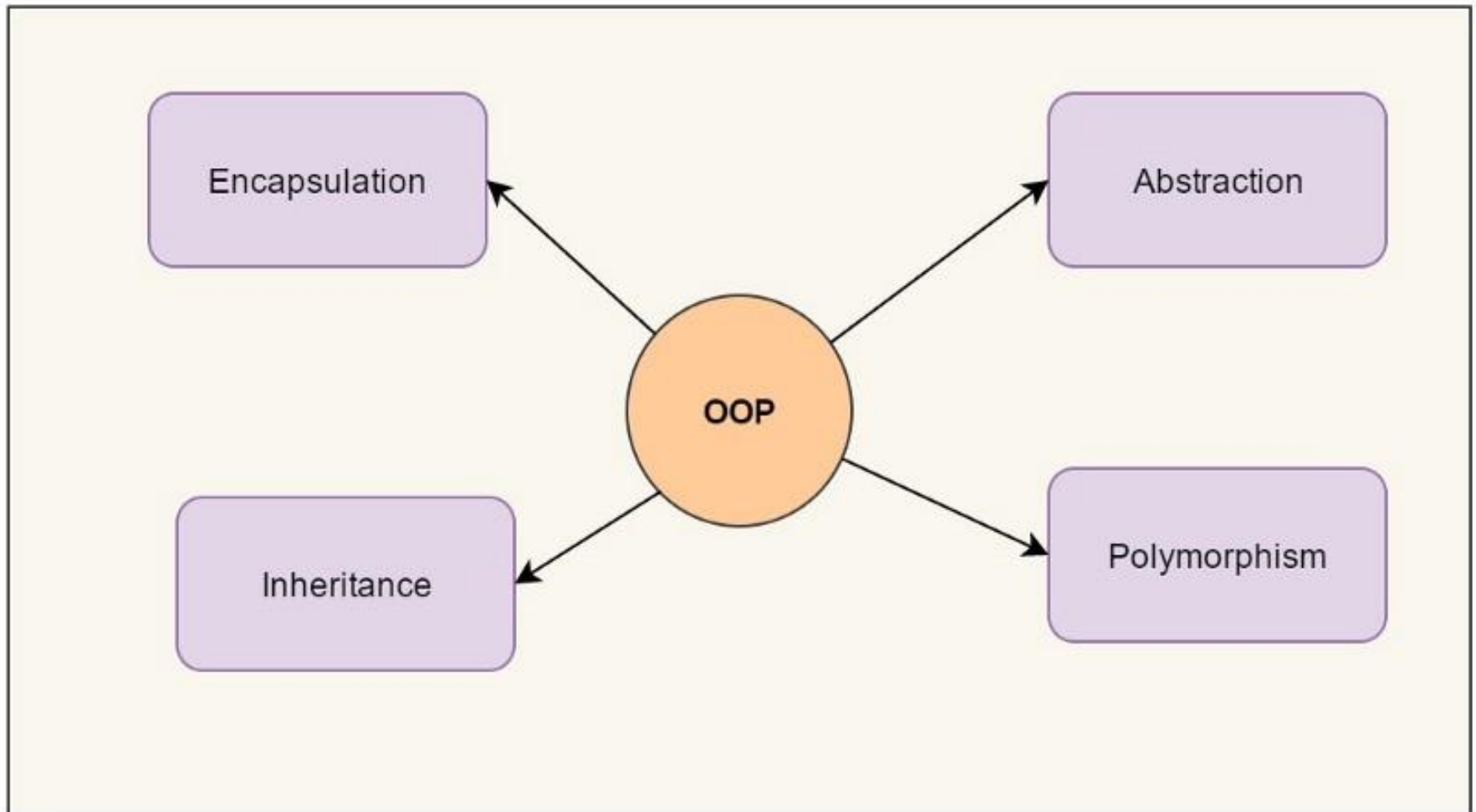


Introduction to Classes and Objects

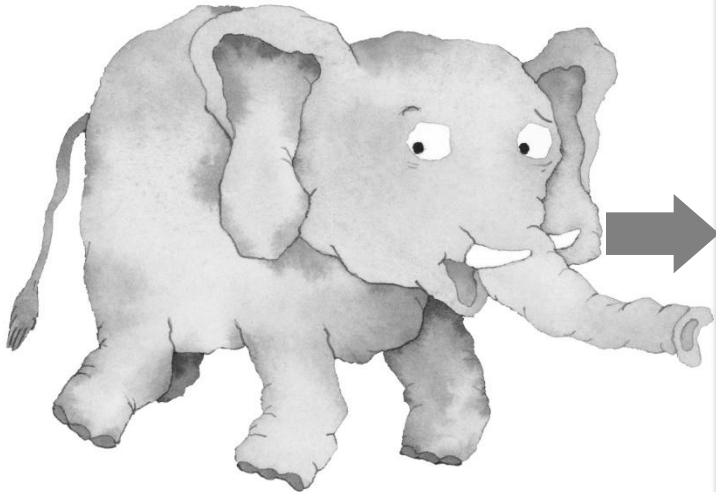
Four Basic Properties of Object Oriented Programming



Four Pillars of Object Oriented Programming

Class

- Abstraction of an object
- Define real world (object) with data and functionality
- Encapsulation



characteristic 1. four legs (data)

characteristic 2. length of nose is about 5 meters (data)

characteristic 3. weight is over 1 tone (data)

characteristic 4. can shower using noise (functionality)

characteristic 5. can pick up a thing (functionality)

Introduction

- Procedural Programming
 - All statements were located in function main
 - Procedures
- Object Oriented Programming
 - Programs will consist of
 - Function main and
 - One or more classes
 - Each containing data members and member functions

Defining a Class With a Member Function

1. A class is a way to bind data and its associated functions together.
2. It allows data (and functions) to be hidden, if necessary, from external use.
3. By defining a class we are creating a new abstract data type that can be treated like any other built-in type.

A class specification has two parts:

1. Class definition.
2. Class function definition.

Syntax for Class Declaration

```
class class_name
```

```
{
```

```
Private:
```

```
Variable declarations;
```

```
Function declarations;
```

```
Public:
```

```
Variable declarations;
```

```
Function declarations;
```

```
};
```

1. The class declaration describes the type and scope of its members.
2. The class function definitions describe how the class functions are implemented.

Basics for Class

1. The class body contains the declaration of variables and functions.
2. These functions and variables collectively called class members.
3. They are usually grouped under two sections , namely private and public to denote which of the members are private and which of them are public.
4. The keyword **private** and **public** are known as **visibility labels**.
5. **Note that these keywords are followed by a colon.**

Private and Public Visibility Labels

1. The class members that have been declared as private can be accessed only from within the class
2. Public members can be accessed from outside the class also.
3. The data hiding(using private declaration)is the key feature of OOP.
4. The use of keyword private is optional.
5. By default , the members of class are private.
6. The variables declared inside the class are known as data members and functions are known as member functions.
7. Only the member functions can have access to the private class members.
8. However, the public members(both data and functions) can be accessed from outside the class.

```
#include<iostream>,  
using namespace std;
```

```
class item  
{  
    int number;  
    float cost;  
  
public:  
    void getdata(int a, float b);  
    void putdata(void);  
};
```

Creating Objects

1. Declaration of class item does not define any object of item but only specifies what they will contain.
2. Once a class has been declared , we can create variables of that type by using the class name (like any other built in type variable).
3. `item x; //` creates a variable x of type item . In C++ class variables are known as objects. We may also declare more than one object in one statement.

`item x,y,z;`

Defining Member Functions

Member functions can be defined in two places:

1. Outside the class definition
2. Inside the class definition

Outside the Class Definition

An important difference between a member function and a normal function is that a member function incorporates a membership ‘identity label’ in the header.

This label tells the compiler which class the function belongs to.

Return-type class-name :: function-name (argument declaration)

{

Function Body

}

A C++ Program with Class

```
#include<iostream>
using namespace std;
class item
{
    int number;
    float cost;

public:
    void getdata(int a, float b);
    void putdata()
    {
        cout << "number : " << number << endl;
        cout << "cost:  " << cost << endl;
    }
};

void item::getdata(int a, float b)
{
    number = a;
    cost = b;
}

int main()
{
    item x;
    cout << "Object x= " << endl;
    x.getdata(100, 525.5);
    x.putdata();
    system ("pause");
    return 0;
}
```

Private Member Function

A private member function can only be called by another function that is a member of its class. Even an object cannot invoke a private function using dot operator.

```
#include<iostream>
using namespace std;
class item
{
    int number;
    float cost;
    void read()
    {
        cout << "Good Morning" << endl;
    }
public:
    void getdata(int a, float b);
    void putdata()
    {
        read();
        cout << "number : " << number << endl;
        cout << "cost:  " << cost << endl;
    }
};
```

```
|void item::getdata(int a, float b)
{
    number = a;
    cost = b;
}
|int main()
{   item x;
    x.read();
    cout << "Object x= " << endl;
    x.getdata(100, 525.5);
    x.putdata();
    system ("pause");
    return 0;
}
```

set Functions and *get* Functions

Software engineering with *set* and *get* functions

1. public member functions that allow clients of a class to set or get the values of private data members
2. *set* functions sometimes called mutators and *get* functions sometimes called accessors
3. Allows the creator of the class to control how clients access private data
4. Should also be used by other member functions of the same class

Accessor and Mutator Function

```
#include<iostream>
#include<conio.h>
using namespace std;
class classone{
private:
    int num;
public:
    void set(int x)
    {
        num = x;
    }
    int get()
    {
        return num;
    }
};
```

```
int main()
{
    classone o1;
    classone o2;
    o1.set(15);
    cout << o1.get()<<endl;
    cout << o2.get();

    cin.ignore();
    cin.clear();
    return 0;
}
```

UML Diagram

- Indicating the return type of an operation
 - Place a colon and the return type after the parentheses following the operation name
- Minus sign used to indicate private members

